Analysis of Searching Algorithms

1. Introduction

This analysis compares three searching algorithms used to find student registration numbers in a large-scale student record system:

- Linear Search
- Recursive Binary Search
- Hash Table Search

The algorithms are evaluated based on time complexity, number of comparisons, preprocessing requirements, and performance as the dataset increases from 1,000 to 1,000,000 records.

2. Time Complexity Analysis

Linear Search

Linear Search checks each element sequentially until the target is found or the list ends.

- Best Case: O(1)
- Average Case: O(n)
- Worst Case: O(n)

Its performance decreases as the number of records increases because it may need to examine many elements.

Recursive Binary Search

Binary Search compares the target with the middle element and repeatedly searches one half of the sorted list.

- Best Case: O(1)
- Average Case: O(log n)
- Worst Case: O(log n)

It is efficient for large datasets but requires the records to be sorted. Sorting an unsorted dataset generally takes O(n log n) time using a comparison-based sorting algorithm.

Hash Table Search

Hash Table Search uses a hash function to locate a registration number in an indexed data structure.

- Best Case: O(1)
- Expected Average Case: O(1)
- Worst Case: O(n)

The expected constant-time lookup depends on a suitable hash function and effective collision management. Building the hash table takes O(n) expected time.

3. Comparison of Algorithms

Algorithm	Best Case	Average Case	Worst Case	Preprocessing
Linear Search	O(1)	O(n)	O(n)	Not required
Recursive Binary Search	O(1)	O(log n)	O(log n)	Sorting required if unsorted
Hash Table Search	O(1)	O(1) expected	O(n)	Hash table construction

4. Theoretical Analysis

As the dataset increases, Linear Search requires more comparisons on average. For one million records, an unsuccessful search can examine all one million records.

Binary Search reduces the search interval by approximately half at every step. For one million sorted records, it requires around 20 comparison levels in the worst case.

Hash Table Search provides expected constant-time lookup, making it suitable for workloads involving frequent searches. However, it requires additional memory and time to construct and maintain the index.

5. Experimental Performance Analysis

The assignment evaluates datasets containing:

- 1,000 records
- 10,000 records
- 100,000 records

- 1,000,000 records

The expected experimental trends are:

1. Linear Search shows increasing search time and comparison counts as dataset size increases.
2. Binary Search maintains a relatively small number of comparisons because it repeatedly halves the search interval.
3. Hash Table Search generally performs a small number of lookup operations, although actual execution time depends on implementation and system conditions.

Actual execution times may vary depending on processor speed, memory, programming language implementation, and system load. Measured results should be taken from the benchmark output generated on the computer used for testing.

6. Theory Versus Experiment

The theoretical complexity describes how the amount of work grows with input size. Experimental measurements show how the algorithms behave in a particular execution environment.

Differences between theoretical and measured performance may occur because of interpreter overhead, memory access, preprocessing, and timing variations.

A fair comparison should distinguish search time from preprocessing time. Binary Search may require sorting, while Hash Table Search requires building an index. These costs should be considered when selecting an algorithm for a real system.

7. Conclusion

The analysis indicates that Linear Search is simple but becomes less efficient for large datasets. Recursive Binary Search provides efficient searching when records are sorted. Hash Table Search offers expected constant-time lookup and is appropriate for read-heavy student record systems.

For one million records and 50,000 searches per hour, an indexed lookup strategy is preferable to repeatedly scanning the entire dataset. In a production database, a suitable persistent database index should be selected and benchmarked according to the system's requirements.